

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion y Diseno Digital

BASES DE DATOS DISTRIBUIDAS

Unidad 3 -- Aplicaciones Distribuidas (ISR-701)

Septimo nivel | Carrera de Ingenieria de Software

Periodo Academico 2026 - 2027

Docente: Ph.D. Gleiston C. Guerrero-Ulloa

Hoja de ruta de la Unidad 3

Tres semanas, tres bloques tematicos

Semana 9

Fundamentos

- Definicion y conceptos
- Caracteristicas
- Las 12 reglas de Date
- Arquitecturas y transparencias
- CAP aplicado a BDD

Semana 10

Diseno físico

- Fragmentacion H, V, mixta
- Algebra relacional formal
- Replicacion sincrona / asincrona
- Modelos de escritura
- Catalogo distribuido

Semana 11

Operacion y NoSQL

- Transacciones distribuidas
- 2PL, 2PC, 3PC
- Deadlock distribuido y MVCC
- PACELC y NoSQL
- Ejercicio practico Banco del Austro

Definicion formal de BDD

Ozsu & Valduriez, 2020

Definicion

Una base de datos distribuida (BDD) es una coleccion de bases de datos multiples, logicamente interrelacionadas, distribuidas sobre una red de computadores. El Sistema de Gestion de Bases de Datos Distribuidas (SGBDD o DDBMS) es el software que gestiona la BDD y hace que la distribucion sea transparente para los usuarios.

Interrelacion logica

Los datos en distintos sitios no son meramente coexistentes. Conforman un esquema global coherente con relaciones entre tablas, integridad referencial y semantica unificada.

Transparencia de la distribucion

La complejidad de la ubicacion fisica, la fragmentacion y la replicacion queda oculta al usuario final y a las aplicaciones cliente. La BDD se percibe como una sola base de datos logica.

Que NO es una base de datos distribuida

Diferenciacion conceptual

1

Base centralizada con acceso remoto

Un unico servidor recibe consultas via red, pero no hay distribucion de datos.

Ejemplo: PostgreSQL en la nube accedido desde sucursales por VPN.

2

Conjunto de bases independientes

Varios servidores que comparten datos por exportacion CSV o procesos batch.

Ejemplo: ERP departamental que envia archivos al sistema corporativo cada noche.

3

Base de datos paralela

Multiples procesadores sobre el MISMO sitio fisico para acelerar consultas.

Ejemplo: Oracle RAC con varios nodos sobre un mismo storage compartido.

Diferencia esencial

La diferencia no radica en la tecnologia de almacenamiento, sino en la distribucion geografica o logica de los datos y en como el sistema gestiona la transparencia de esa distribucion.

Vocabulario tecnico fundamental (1/2)

Terminos imprescindibles para el resto del modulo

Sitio o nodo

Unidad logica de almacenamiento y procesamiento. Servidor fisico, contenedor Docker, maquina virtual o pod de Kubernetes (K8s).

Esquema global

Descripcion logica completa de la BDD: relaciones, atributos, restricciones de integridad. Es lo que ve la aplicacion.

Esquema local

Descripcion fisica de cada sitio: los fragmentos o replicas que ese sitio almacena efectivamente.

Fragmento

Porcion de una relacion del esquema global asignada a un sitio. Puede ser horizontal (filas), vertical (columnas) o mixto.

Replica

Copia identica de un fragmento o relacion completa que reside en mas de un sitio. Mejora disponibilidad y latencia de lectura.

Vocabulario tecnico fundamental (2/2)

Operacion, consistencia y red

Catalogo distribuido

Estructura de metadatos que indica donde reside cada fragmento y replica. Habilita la transparencia de fragmentacion y ubicacion.

Transaccion distribuida

Unidad logica de trabajo que accede a datos en varios sitios. Debe cumplir ACID globalmente, lo que exige protocolos como el commit en dos fases (2PC).

Procesamiento distribuido de consultas

Tecnicas que transforman una consulta global en un plan que combina operaciones locales y transferencias por red, minimizando costo.

Modelo de consistencia

Garantia formal del sistema sobre el orden y la visibilidad de operaciones concurrentes. Va de serializabilidad estricta a consistencia eventual.

Tolerancia a particiones de red

Capacidad del sistema de continuar operando cuando la red impide la comunicacion entre subconjuntos de sitios. Es la P del teorema CAP.

Características de las BDD

Cuatro fortalezas estructurales

1

Distribucion fisica y logica

Datos en sitios geografica o logicamente separados. Intraciudad, interurbana o intercontinental. El sistema mantiene la ilusion de un unico almacen.

2

Autonomia local

Cada sitio opera independientemente cuando la red falla. Las consultas y transacciones locales no se ven afectadas por fallos externos.

3

Crecimiento incremental

Anadir sitios sin reescribir aplicaciones ni reorganizar masivamente. Escalabilidad horizontal: anada nodos en lugar de comprar un servidor mas grande.

4

Mayor disponibilidad

La replicacion permite que la caida de un sitio no implique perdida de servicio. Con consenso por mayoria (Raft, Paxos) se tolera la perdida de un nodo de tres sin interrupcion.

Características y compromisos inherentes

Las ventajas no son gratuitas

Mas ventajas

Mejor rendimiento localizado

Localidad de referencia: los datos se ubican donde se usan con mayor frecuencia.

Heterogeneidad potencial

Distintos sitios pueden ejecutar distintos motores SGBD, integrando sistemas legados sin migrar.

Compromiso CAP explicito

Cualquier BDD declara su posicion: ante particion, sacrifica consistencia (AP) o disponibilidad (CP).

Compromisos inherentes

- Complejidad de coordinacion entre sitios
- Latencia de red en operaciones cross-site
- Dificultad para mantener integridad referencial
- Problemas de consistencia ausentes en centralizados
- Despliegue, monitoreo y debugging mas complejos
- Costo operativo mayor (mas hardware, mas personal capacitado)

Las 12 reglas de Date para SGBDD (1/2)

Estandar de referencia desde 1987

R1

Autonomia de sitio local

Cada sitio gestiona su seguridad e integridad sin depender de otros.

R2

Sin dependencia de un sitio central

Ningun nodo cuya caida paralice el sistema completo.

R3

Operacion continua

El sistema sigue disponible aunque fallen nodos o se haga mantenimiento.

R4

Independencia de ubicacion

El usuario no necesita conocer en que sitio reside un dato para acceder a el.

R5

Independencia de fragmentacion

Consultas escritas contra el esquema global sin importar como esta dividida cada relacion.

R6

Independencia de replicacion

Las aplicaciones operan como si existiera una unica copia, aunque internamente haya varias.

Las 12 reglas de Date para SGBDD (2/2)

Independencias y procesamiento distribuido

R7

Procesamiento de consultas distribuido

El optimizador maneja consultas multi-sitio y elige el plan mas eficiente.

R8

Manejo de transacciones distribuidas

Las propiedades ACID se garantizan globalmente con protocolos como 2PC.

R9

Independencia de hardware

El sistema funciona sobre maquinas con distinta arquitectura.

R10

Independencia de sistema operativo

Los sitios pueden correr Linux, Windows o macOS sin afectar el funcionamiento global.

R11

Independencia de red

Protocolos de comunicacion (TCP/IP, RPC, gRPC) transparentes al usuario.

R12

Independencia de SGBD

Distintos sitios pueden usar motores distintos, siempre que se cumpla un estandar comun (SQL, JDBC).

Tres arquitecturas de SGBDD

Diferenciadas por autonomia y coordinacion

Cliente-Servidor

Un servidor centralizado gestiona la BDD; los clientes envían solicitudes vía red. En la variante distribuida moderna el servidor es un cluster con replicación primaria-secundaria.

+ Ventaja

Simplicidad operativa y facilidad de gestión

- Limitación

Cuello de botella y punto único de fallo (mitigado por replicación)

Ejemplos

PostgreSQL con streaming replication, MySQL con replicación asíncrona

Federada

Múltiples SGBD locales preexistentes se interconectan mediante una capa que ofrece una vista global unificada. Cada sitio conserva autonomía y esquema local.

+ Ventaja

Integración de sistemas heterogéneos sin migrarlos

- Limitación

Complejidad de coherencia semántica y optimización limitada

Ejemplos

PostgreSQL con Foreign Data Wrappers (FDW), Oracle Federated, Denodo

Par a Par (P2P)

Todos los nodos tienen el mismo rol: pueden recibir consultas, almacenar datos y participar en el consenso. No existe coordinador central. Comunicación vía protocolos gossip.

+ Ventaja

Escalabilidad horizontal prácticamente ilimitada

- Limitación

Máxima complejidad de coordinación: consenso (Raft, Paxos) indispensable

Ejemplos

Apache Cassandra, ScyllaDB, Riak, CockroachDB

Las ocho transparencias en SGBDD

El principio rector del diseno

Transparencia: ocultar al usuario y a la aplicacion la complejidad de la distribucion para que el sistema se perciba como si fuera centralizado.

1

De red (acceso)

El usuario no conoce topologia ni protocolos.

2

De ubicacion

No sabe en que sitio fisico residen los datos.

3

De fragmentacion

No sabe como se dividio cada relacion.

4

De replicacion

No sabe cuantas copias del dato existen.

5

De concurrencia

Las transacciones concurrentes lucen seriales.

6

De fallos

Los fallos de nodos se manejan internamente.

7

De rendimiento

El optimizador elige el plan mas eficiente.

8

De migracion

Los datos se mueven sin cambios en aplicaciones.

Homogeneidad versus heterogeneidad

Dos familias segun la uniformidad de los sitios

BDD Homogenea

Todos los sitios usan el mismo motor SGBD, el mismo esquema y la misma version de software. Simplifica coordinacion, transacciones distribuidas y optimizador. Todos los sitios hablan el mismo idioma.

Ejemplos representativos

- CockroachDB cluster
- Cassandra cluster
- MongoDB Sharded
- PostgreSQL + Citus

Bajo costo de integracion, alta eficiencia en optimizacion distribuida.

BDD Heterogenea

Distintos sitios ejecutan motores SGBD distintos: PostgreSQL en un sitio, Oracle en otro, MySQL en un tercero. Requiere capa de federacion que traduzca consultas y reconcilie tipos, semantica y modelos.

Ejemplos representativos

- PostgreSQL FDW hacia Oracle
- Denodo Data Virtualization
- Apache Drill
- Presto / Trino

Permite integrar sistemas legados sin migrarlos; mayor complejidad de mappings.

Fragmentacion horizontal

Division por filas con algebra relacional

Definicion formal

Divide las FILAS (tuplas) de una relacion entre los sitios. Cada fragmento contiene un subconjunto de tuplas que satisfacen un predicado de seleccion.

Operador del algebra relacional:

$F_i = \sigma_{\pi_i}(R), i = 1..n$

Reconstruccion:

$R = F_1 \text{ UNION } F_2 \text{ UNION } \dots \text{ UNION } F_n$

Variante: fragmentacion horizontal derivada

Los fragmentos de una relacion R2 se definen segun los de otra R1 con la que esta relacionada por clave foranea. Mantiene la localidad de los JOIN.

Ejemplo: PEDIDOS por ciudad

Relacion original: PEDIDOS(id, cliente, ciudad, monto)

$F1 = \sigma_{\{ciudad='Cuenca'\}}(PEDIDOS)$

-> Nodo Cuenca

$F2 = \sigma_{\{ciudad='Quito'\}}(PEDIDOS)$

-> Nodo Quito

Reconstruccion: $PEDIDOS = F1 \text{ UNION } F2$

Beneficio clave

Los pedidos de clientes de Cuenca se procesan localmente en el nodo Cuenca, reduciendo latencia. Una consulta `WHERE ciudad='Cuenca'` se redirige solo al nodo Cuenca, evitando red.

Fragmentacion vertical

Division por columnas con clave en cada fragmento

Definicion formal

Divide las COLUMNAS (atributos) de la relacion. Cada fragmento incluye la clave primaria (para permitir la reconstruccion mediante JOIN) y un subconjunto de los demas atributos.

Operador del algebra relacional:

$FV_i = \pi_{\{PK, A_i\}}(R)$

Reconstruccion:

$R = FV_1 \text{ JOIN } FV_2 \text{ JOIN } \dots \text{ JOIN } FV_n$

Diseno optimo: analisis de afinidad

Construir matriz que mide cuantas veces dos atributos se consultan juntos. Algoritmo Hoffer-Severance agrupa los atributos con alta afinidad en el mismo fragmento.

Ejemplo: EMPLEADOS por sensibilidad

EMPLEADOS(id, nombre, cargo, salario, depart, telefono)

$FV1 = \pi_{\{id, nombre, cargo, depart\}}(EMP)$

-> Nodo RRHH

$FV2 = \pi_{\{id, salario, telefono\}}(EMP)$

-> Nodo Nomina (cifrado)

Reconstruccion: $EMP = FV1 \text{ JOIN}_{id} FV2$

Atencion: la clave primaria DEBE

estar en TODOS los fragmentos verticales, de lo contrario no es posible reconstruir la tabla. Es la causa mas comun de violacion de la regla de reconstruccion en disenos verticales.

Fragmentacion mixta y reglas de correctitud

Combinacion y validacion de cualquier esquema

Fragmentacion mixta (hibrida)

Combina horizontal y vertical. Aplicar primero una fragmentacion horizontal y luego sobre cada fragmento horizontal una vertical -- o viceversa.

Ejemplo: VENTAS

`VENTAS(id, vendedor, region, producto, monto)`

Paso 1 - Horizontal por region:

$FH1 = \sigma_{region='Sierra'}(VENTAS)$

$FH2 = \sigma_{region='Costa'}(VENTAS)$

Paso 2 - Vertical sobre FH1:

$FH1V1 = \pi_{id, vendedor}(FH1) \rightarrow Ventas$

$FH1V2 = \pi_{id, producto, monto}(FH1) \rightarrow Contabilidad$

Tres reglas de correctitud

C1

Compleitud

Todo dato de la relacion original esta en al menos un fragmento. Union completa.

C2

Reconstruccion

La relacion se reconstruye con operadores del algebra: UNION para H, JOIN para V.

C3

Disjuncion

En H: ningun dato aparece en dos fragmentos. En V: solo la clave primaria se repite.

Replicación sincrona versus asincrona

Compromiso fundamental entre consistencia y latencia

SINCRONA

La escritura se confirma al cliente solo despues de que todas las replicas -- o un quorum -- hayan persistido el cambio.

Consistencia: **Fuerte**

Latencia de escritura: **Alta**

Disponibilidad ante fallo: **Reducida**

Riesgo de perdida: **Ninguno**

Caso de uso: **Banca, inventario**

Tecnologia ejemplo: **PostgreSQL synchronous standby**

ASINCRONA

La escritura se confirma al cliente inmediatamente despues de persistir en la primaria; las replicas se actualizan despues, en segundo plano.

Consistencia: **Eventual**

Latencia de escritura: **Baja**

Disponibilidad ante fallo: **Alta**

Riesgo de perdida: **Si (lag)**

Caso de uso: **Redes sociales, logs**

Tecnologia ejemplo: **MySQL async replica**

Modelos de escritura en sistemas replicados

Quien acepta escrituras y como se acuerda

Primario-Secundario

Una replica designada como primaria es la unica autorizada para aceptar escrituras. Las secundarias propagan las escrituras y solo sirven lecturas.

Pro: Consistencia fuerte natural

Contra: Primario es cuello de botella y SPOF

MySQL, MongoDB Replica Set, PostgreSQL streaming

Multi-Primario

Varias replicas aceptan escrituras concurrentes. Mejora disponibilidad de escritura, pero introduce conflictos que requieren politicas de resolucion.

Pro: Alta disponibilidad de escritura

Contra: Conflictos: last-write-wins, vector clocks, CRDTs

MySQL Group Replication, Galera Cluster, Couchbase XDCR

Sin lider (leaderless)

Cliente escribe en varios nodos en paralelo; escritura exitosa si W confirman. Lecturas consultan R nodos y comparan. Quorum: $W + R > N$ garantiza consistencia.

Pro: Maxima disponibilidad y escalabilidad

Contra: Logica de cliente compleja; consistencia ajustable

Apache Cassandra, Riak, Amazon DynamoDB

Quorum: con $N=5$ replicas, una configuracion comun es $W=3, R=3$ ($3+3=6 > 5$): tolera la falla de hasta 2 nodos manteniendo consistencia

Catalogo distribuido

Los metadatos que habilitan la transparencia

Definicion

Estructura de metadatos que describe DONDE residen los fragmentos y replicas de cada relacion del esquema global. Cuando una consulta llega al SGBDD, este consulta primero el catalogo para enrutarla a los sitios correctos.

1

Centralizado

Un solo nodo guarda todos los metadatos. Simple pero introduce SPOF: si el nodo del catalogo cae, ninguna consulta global se enruta.

Pro: Simplicidad

Contra: SPOF critico

2

Completamente replicado

Copia identica en cada nodo. Enrutamiento local sin red. Costoso al actualizar: cada cambio requiere transaccion distribuida sobre el catalogo.

Pro: Latencia minima

Contra: Costo de actualizacion

3

Parcialmente replicado

Cada nodo guarda metadatos locales mas info de los nodos con los que mas interactua. Balance entre disponibilidad y costo. Cassandra y CockroachDB lo adoptan.

Pro: Balance optimo

Contra: Complejidad de gestion

Procesamiento de consultas distribuidas

Cuatro fases del SGBDD ante una consulta SQL

1

Descomposicion

Traduce SQL a expresion del algebra relacional sobre el esquema global. Valida sintaxis, semantica y vistas.

2

Localizacion

Convierte la consulta global en una consulta sobre fragmentos especificos usando el catalogo.

3

Optimizacion global

Elige el plan mas eficiente: donde hacer JOINS (semi-joins, bloom-joins), si replicar fragmentos, en que orden.

4

Ejecucion

Despacha sub-consultas, recibe resultados parciales, los combina segun el plan. Coordinador puede ser dinamico.

Tecnicas de optimizacion clave

Filtrado temprano:

Selecciones y proyecciones antes de transferir

Semi-join:

Transferir solo claves para reducir JOIN

Bloom-join:

Filtros de Bloom para reducir semi-join

Replicacion temporal:

Copiar fragmento pequeno para evitar JOIN

Partition-wise join:

JOIN entre co-particionados localmente

Cache de resultados:

Reutilizar resultados entre consultas

Transacciones distribuidas: ACID en escala

Cuatro propiedades, cuatro desafios

Una transaccion distribuida es una unidad logica de trabajo que accede a datos en multiples sitios y debe cumplir las propiedades ACID de forma GLOBAL.



Atomicidad

Todo o nada en todos los sitios

Desafio distribuido

Requiere protocolo 2PC para coordinar



Consistencia

Estado valido tras la transaccion

Desafio distribuido

Requiere acuerdo global de validez



Aislamiento

Sin efectos laterales visibles

Desafio distribuido

Requiere bloqueos distribuidos (2PL) o MVCC



Durabilidad

Cambios persisten ante fallos

Desafio distribuido

Requiere WAL en cada nodo participante

Two-Phase Locking (2PL)

Protocolo de bloqueo en dos fases para serializabilidad

Definicion

Protocolo clasico para garantizar la SERIALIZABILIDAD de transacciones concurrentes. Cada transaccion atraviesa dos fases.

Fase 1: Crecimiento

La transaccion SOLO puede adquirir bloqueos. No puede liberar ninguno todavia.

Fase 2: Decrecimiento

Tras liberar el primer bloqueo, NO puede adquirir mas. Solo libera los que ya posee.

Variante 2PL Estricto: mantiene todos los bloqueos hasta COMMIT o ROLLBACK. Evita lecturas sucias.

Tipos de bloqueos

S (Compartido)

Uso: Lectura

Compat: Si con S, No con X

X (Exclusivo)

Uso: Escritura

Compat: No con nada

U (Update)

Uso: Previo a X

Compat: Si con S, No con X

IS / IX (Intencion)

Uso: Jerarquias

Compat: Para anidados

En BDD: gestor centralizado o distribuido con coordinacion de deadlocks.

Two-Phase Commit (2PC)

El protocolo estandar de atomicidad distribuida

Garantiza la ATOMICIDAD: todos los participantes confirman o todos abortan. Coordinado por un nodo coordinador que dirige a varios participantes.

FASE 1 - Preparacion (Voting)

1. Coordinador envia PREPARE a todos los participantes
2. Cada participante ejecuta la parte local
3. Escribe PREPARED en su log WAL
4. Responde VOTE-COMMIT si pudo, VOTE-ABORT si fallo

FASE 2 - Decision (Commit/Abort)

1. Si TODOS votaron COMMIT: coordinador escribe COMMIT en su log
2. Envia GLOBAL-COMMIT a todos los participantes
3. Si ALGUNO voto ABORT o no respondio: envia GLOBAL-ABORT
4. Cada participante aplica la decision y libera bloqueos

Limitacion del 2PC: bloqueo y 3PC

Por que los sistemas modernos prefieren consenso



El 2PC es un protocolo BLOQUEANTE

Si el coordinador falla DESPUES de la fase de preparacion pero ANTES de comunicar la decision, los participantes quedan en estado PREPARED reteniendo bloqueos, sin saber si confirmar o abortar. Pueden quedar bloqueados indefinidamente.

3PC: Three-Phase Commit

Extiende 2PC con una fase PRE-COMMIT intermedia. Tras votacion exitosa, el coordinador envia PRE-COMMIT; solo despues envia COMMIT final.

Si el coordinador falla durante pre-commit, los participantes pueden elegir nuevo coordinador que decida sin ambigüedad. PERO: asume modelo de red sincrono (mensajes con retardo acotado), rara vez cumplido. Casi no se usa en produccion.

Alternativa moderna: Consenso (Raft / Paxos)

Algoritmos de consenso distribuido que toleran fallos sin asumir sincronia. Raft (Ongaro & Ousterhout 2014) es el mas adoptado.

Usado por etcd (Kubernetes), CockroachDB, MongoDB (Replica Set elections), Apache Kafka KRaft, HashiCorp Consul. Sustituye al 2PC en sistemas modernos al replicar el log de decisiones por mayoria.

Deadlock distribuido

Deteccion y resolucion en grafos de espera multi-nodo

Definicion

Dos o mas transacciones esperan indefinidamente que la otra libere un recurso. En entornos distribuidos el grafo de espera se extiende entre nodos, dificultando su deteccion.

1

Prevencion por ordenamiento

Bloqueos siempre en el mismo orden global (alfabetico por recurso). Elimina deadlocks por construccion.

Contra: Reduce paralelismo

2

Prevencion por timestamps

Wait-Die: aborta la transaccion mas joven. Wound-Wait: aborta la mas vieja. Ambos previenen deadlock.

Contra: Mas abortos por contencion

3

Deteccion por grafo de espera

Sistema construye grafo dirigido $T_i \rightarrow T_j$ si T_i espera recurso que T_j tiene. Ciclo = deadlock. Algoritmo elige victima.

Contra: Costo de mantener el grafo

4

Deteccion por timeout

Si transaccion espera mas de un umbral, se asume deadlock y se aborta. Simple pero genera falsos positivos.

Contra: Falsos positivos en cargas altas

MVCC: Multi-Version Concurrency Control

Alternativa moderna al 2PL clasico

Concepto

Cada escritura crea una NUEVA VERSION del registro etiquetada con el identificador de la transaccion. Cada transaccion ve una SNAPSHOT consistente del estado del sistema al momento en que inicio. Las lecturas NUNCA bloquean escrituras ni viceversa.

Implementacion en PostgreSQL

Cada tupla almacena dos campos:

xmin: id de transaccion que la inserto

xmax: id de transaccion que la elimino o reemplazo

Una transaccion T ve una version si: $xmin \leq T$ AND ($xmax > T$ OR $xmax = NULL$).
Versiones obsoletas se eliminan con VACUUM.

Comparacion MVCC vs 2PL

Bloqueos en lectura	Ninguno	Si (S lock)
Versiones por registro	Multiples	Una sola
Lectura sin bloqueo	Si	Bloqueada si X
Mantenimiento	VACUUM/cleanup	Liberacion de bloqueos
Usado por	PostgreSQL, Cockroach	MySQL clasico, SQL Srv

Teorema CAP aplicado a BDD

Elección explícita entre Consistencia y Disponibilidad

Teorema CAP (Brewer 2000, formalizado 2002)

Un sistema distribuido no puede garantizar simultaneamente las tres propiedades:



Consistency

Toda lectura recibe el dato mas reciente o un error



Availability

Toda petición recibe respuesta (no error), aunque no sea la mas reciente

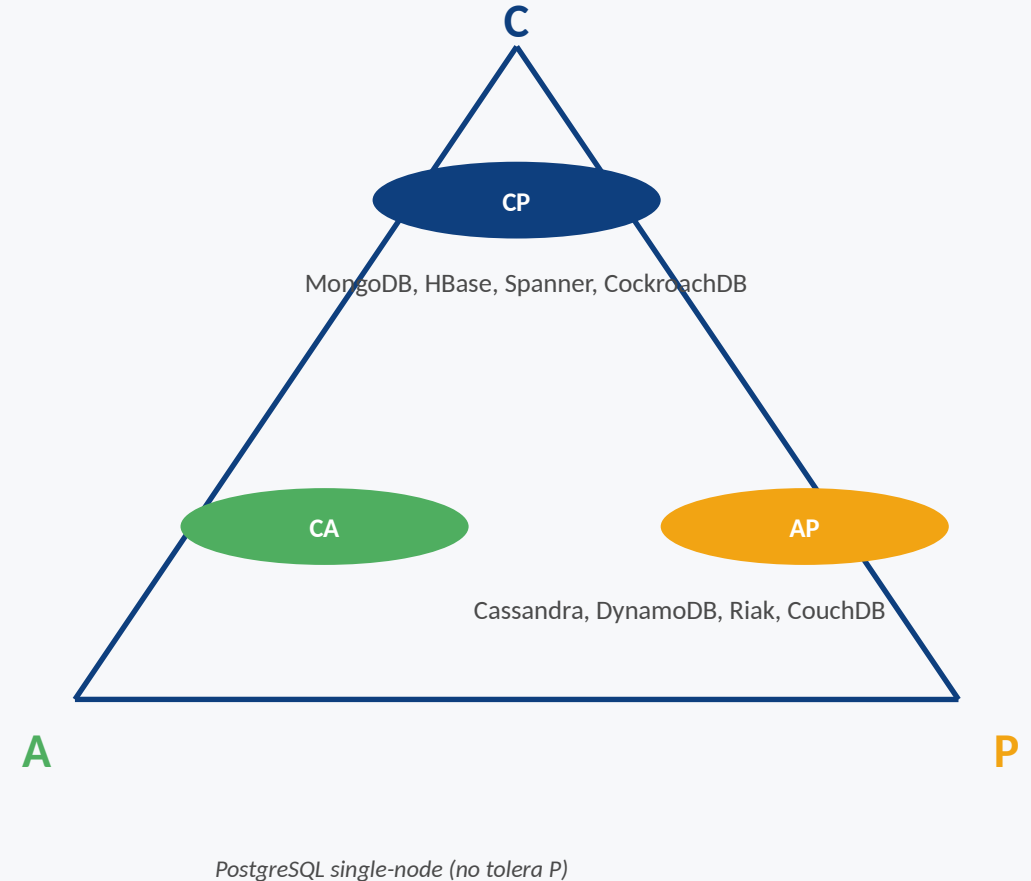


Partition tolerance

El sistema sigue operando cuando la red presenta particiones

Como las particiones son inevitables, el verdadero compromiso es CP vs AP: ante una partición, sacrificar consistencia o disponibilidad.

Posicionamiento de SGBDD



Modelo PACELC

Refinamiento moderno del teorema CAP

PACELC (Abadi 2012)

Si hay **P**articion: elegir entre **A**vailability y **C**onsistency. **E**lse (sin particion): elegir entre **L**atency y **C**onsistency.

Clasificacion PACELC de sistemas reales

Sistema	Particion	Sin part.	Caso de uso típico
Google Spanner	PC	EC	Banca global, Google Ads
CockroachDB	PC	EC	Banca, retail multi-region
MongoDB (writeConcern majority)	PC	EC	Catalogos con consistencia
HBase	PC	EC	Big Data Hadoop
Apache Cassandra	PA	EL	IoT, logs, recomendaciones
Amazon DynamoDB	PA	EL	Sesiones, carritos AWS
Riak	PA	EL	Bases clave-valor distribuidas

Las cuatro familias NoSQL

Modelos alternativos al relacional

Documentales

Documentos JSON / BSON en colecciones. Cada documento puede tener esquema distinto. Anidacion natural elimina muchos JOIN.

Ejemplos: MongoDB, CouchDB, DocumentDB

Caso de uso: Catalogos, perfiles, configuraciones

Clave-Valor

Asociacion clave-valor (string o blob). Latencias submilisegundo, enorme rendimiento. Operaciones limitadas a get/put/delete.

Ejemplos: Redis, Memcached, DynamoDB, Riak

Caso de uso: Caches, sesiones, carritos

Columnares (Wide Column)

Almacenamiento por columnas. Cada fila puede tener columnas distintas. Compresion y consultas analiticas optimizadas.

Ejemplos: Cassandra, HBase, Bigtable, ScyllaDB

Caso de uso: Series de tiempo, IoT, logs

Grafos

Nodos y aristas con propiedades. Optimizan consultas que recorren relaciones de varios saltos. Lenguajes Cypher, Gremlin.

Ejemplos: Neo4j, JanusGraph, Neptune, TigerGraph

Caso de uso: Redes sociales, fraude, rutas

Sistemas reales (1/2): Spanner, CockroachDB, Cassandra

Tres BDD insignia del estado del arte

Google Spanner

Familia: Relacional global

PACELC: PC/EC

Consenso: Paxos + TrueTime

Combina escalabilidad NoSQL con ACID y SQL del mundo relacional. Innovación clave: TrueTime, API de tiempo con incertidumbre garantizada gracias a relojes atómicos y GPS en cada centro de datos.

CockroachDB

Familia: Relacional distribuida

PACELC: PC/EC

Consenso: Raft

SGBDD relacional open source para escalabilidad horizontal y resistencia. Compatible con protocolo PostgreSQL. Fragmenta automáticamente en rangos de 64MB replicados con Raft. Serializabilidad estricta global.

Apache Cassandra

Familia: NoSQL columnar

PACELC: PA/EL

Consenso: Gossip + Hinted Handoff

NoSQL columnar P2P sin líder para escalabilidad masiva y alta disponibilidad. Replicación con quorum configurable (N, W, R). Usado por Netflix (recomendaciones), Apple (mensajería), Instagram (mensajes).

Sistemas reales (2/2): DynamoDB, MongoDB, PostgreSQL

Tres protagonistas del ecosistema actual

Amazon DynamoDB

Familia: NoSQL clave-valor

PACELC: PA/EL

Consenso: Quorum

Servicio gestionado de AWS con escalado automatico, latencias submilisegundo y replicacion multi-region opcional (Global Tables). Hash consistente y sharding automatico transparente.

MongoDB

Familia: Documental

PACELC: PC/EC o PA/EL (config.)

Consenso: Raft (oplog)

Replica Sets con failover automatico y Sharded Clusters coordinados por mongos y config servers. Soporta ACID multi-documento desde 4.0. Consistencia ajustable via writeConcern y readConcern.

PostgreSQL + Citus

Familia: Relacional + sharding

PACELC: PC/EC

Consenso: 2PC + WAL

No nativamente distribuido pero soporta streaming replication, logical replication, Foreign Data Wrappers para federacion, y Citus para sharding horizontal. Base preferida para BDD heterogeneas y ecosistemas SQL maduros.

Recuperacion ante fallos

Seis mecanismos que sostienen la durabilidad

1

Write-Ahead Log (WAL)

Cada sitio mantiene log local de todas las operaciones, persistido en disco antes de aplicar cambios. ARIES como algoritmo de referencia.

2

Checkpoints

Cada sitio escribe periodicamente el estado de transacciones activas, reduciendo el tiempo de recuperacion al limitar log replicable.

3

Logging del coordinador (2PC)

El coordinador escribe COMMIT o ABORT global ANTES de comunicarlo. Si falla, los participantes recuperan la decision del log.

4

Replicacion sincrona como failover

Si una replica falla, otra toma su lugar sin perdida si la replicacion fue sincrona o casi sincrona.

5

Snapshot + Point-In-Time Recovery

Restaurar el sistema a un instante exacto del pasado combinando snapshot completo con WAL posterior. Critico para errores logicos.

6

Consenso (Raft, Paxos)

El log replicado por consenso reemplaza al log local: cualquier nodo con el log puede reconstruir el estado. CockroachDB y etcd lo usan.

Ejercicio Practico

CASO BANCO DEL AUSTRO

BDD homogenea federada con PostgreSQL 16, Docker Compose, Spring Boot 3.x y Java 21

Java 21

Spring Boot 3.x

PostgreSQL 16

Docker Compose

Maven

Equipos AGLS, BCEL, FUVV y ACC: este ejercicio es la base de su entrega PFC E3

Banco del Austro: requisitos y analisis CAP

Del problema de negocio a la decision tecnica

Requisitos no funcionales

- Consultas de saldo deben responder en menos de 200 ms desde cualquier oficina
- Transferencias bancarias deben cumplir ACID estricto
- La caida de una oficina no debe afectar la operacion de las demas
- Datos de clientes compartidos entre oficinas (acceso multi-sucursal)
- Datos de transacciones locales son propios de cada oficina

Decision CAP



Las transferencias requieren consistencia fuerte (ACID).

Ante una particion de red, preferimos BLOQUEAR antes que transferir dinero incorrectamente.

PACELC: PC / EC

Diseno de la distribucion

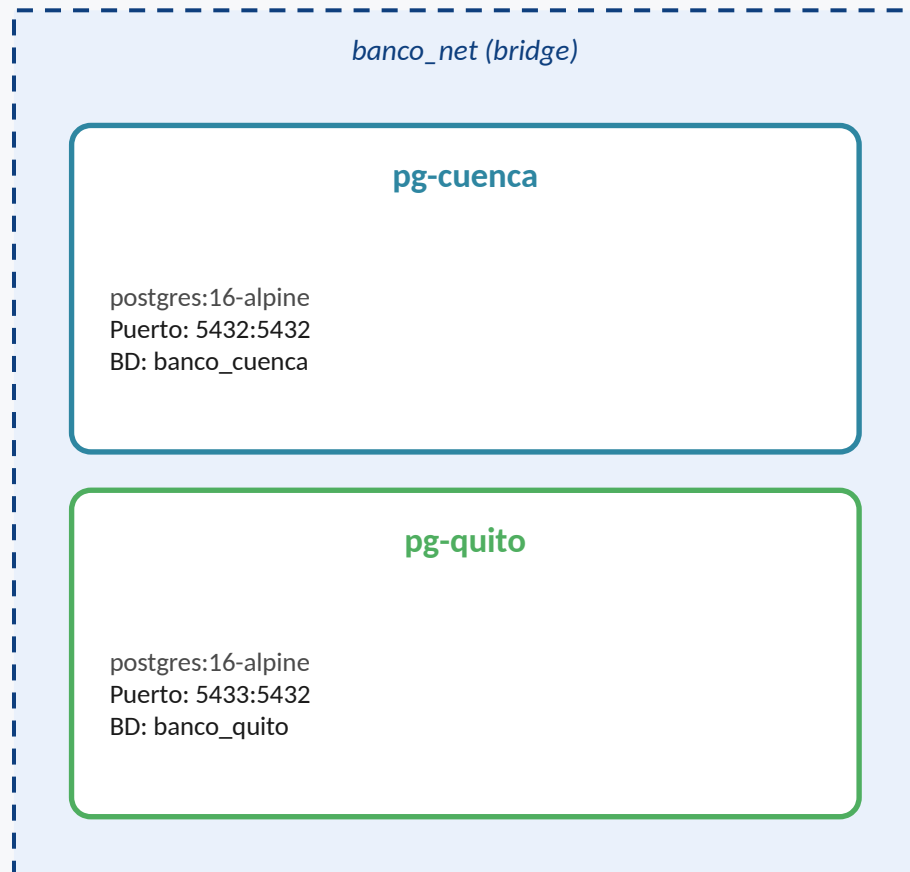
Tres tablas, tres estrategias

CLIENTES REPLICADA	Estrategia Tabla compartida entre todos los nodos. Lectura frecuente desde cualquier oficina; escritura infrecuente.	Justificacion El cliente del Banco del Austro puede operar desde cualquier sucursal. Localmente leer al cliente debe ser inmediato.
CUENTAS FRAGMENTADA HORIZONTAL + REPLICADA	Estrategia Fragmentos por oficina (F_Cuenca, F_Quito) con replicacion en la sede central.	Justificacion Localidad para operaciones del titular en su oficina principal, pero la sede puede auditar todo.
TRANSACCIONES FRAGMENTADA HORIZONTAL	Estrategia Fragmentos por oficina, sin replicacion. Log de operaciones locales.	Justificacion Cada oficina mantiene su historial. Si se necesita reporte global, se hace via union distribuida.

Paso 1: levantamiento del cluster con Docker Compose

Dos nodos PostgreSQL 16 en contenedores

Arquitectura del cluster



docker-compose-bdd.yml (extracto)

services:

pg-cuenca:

```
image: postgres:16-alpine
container_name: pg_cuenca
environment:
  POSTGRES_DB: banco_cuenca
  POSTGRES_USER: banco
  POSTGRES_PASSWORD: banco123
ports:
  - "5432:5432"
volumes:
  - pg_cuenca_data:/var/lib/postgresql/data
  - ./sql/init_cuenca.sql:...
networks: [banco_net]
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U banco"]
```

Paso 2: catalogo distribuido en Java

Componente Spring que enruta segun la distribucion

CatalogoDistribuido.java

```
package ec.uteq.banco.catalogo;

import org.springframework.stereotype.Component;
import java.util.*;

@Component
public class CatalogoDistribuido {

    public enum Fragmento {
        CUENTAS_CUENCA("cuencaDataSource", "Cuenca"),
        CUENTAS_QUITO("quitoDataSource", "Quito");
        ...
    }

    public Optional<Fragmento> resolverPorNumeroCuenta(String n) {
        String prefijo = n.substring(0, 2);
        return switch (prefijo) {
            case "22" -> Optional.of(Fragmento.CUENTAS_CUENCA);
            case "17" -> Optional.of(Fragmento.CUENTAS_QUITO);
            default -> Optional.empty();
        };
    }
}
```

Paso 3: transferencia cross-site con 2PC manual

El nucleo operativo del Banco del Austro

TransferenciaService.transferir() -- extracto clave

```
// FASE 1 - PREPARACION: debitar origen, acreditar destino
cOrig.setAutoCommit(false); cDest.setAutoCommit(false);
if (!debitar(cOrig, cuentaOrigen, monto)) {
    rollbackAmbos(cOrig, cDest);
    return Map.of("status", "ABORT", "mensaje", "Saldo insuficiente");
}
if (!acreditar(cDest, cuentaDestino, monto)) {
    rollbackAmbos(cOrig, cDest);
    return Map.of("status", "ABORT", "mensaje", "Cuenta destino inexistente");
}
registrarTransaccion(cOrig, cuentaOrigen, cuentaDestino, monto);
// FASE 2 - COMMIT: ambos confirman atomicamente
cOrig.commit();
cDest.commit();
log.info("[2PC] Transferencia OK: {} -> {} por {}", cuentaOrigen, cuentaDestino, monto);
return Map.of("status", "COMMIT", ...);
```

Paso 4: API REST y ejecucion completa

Endpoints expuestos y pruebas con curl

Endpoints expuestos

GET /api/banco/saldo/{numero}

Consulta saldo (enrutado por catalogo)

GET /api/banco/clientes

Union distribuida de clientes

GET /api/banco/cuentas/{oficina}

Cuentas por oficina (fragmento local)

POST /api/banco/transferencia

Transferencia cross-site con 2PC

Ejecucion paso a paso

```
# 1. Levantar contenedores
docker compose -f docker-compose-bdd.yml up -d

# 2. Verificar estado healthy
docker compose ps

# 3. Ejecutar la aplicacion Spring Boot
./mvnw spring-boot:run

# 4. Probar saldo (enrutamiento por catalogo)
curl http://localhost:8080/api/banco/saldo/2201000001
curl http://localhost:8080/api/banco/saldo/1701000001

# 5. Transferencia cross-site con 2PC
curl -X POST "http://localhost:8080/api/banco/transferencia?origen=2201000001&destino=1701000001&monto=100"

# 6. Probar falla: detener nodo Quito
docker stop pg_quito
# Repetir transferencia => debe responder ABORT
```

Resultados esperados y extension

Validacion conceptual y reto adicional

Comportamientos validados



Transparencia de fragmentacion

Las consultas se enrutan al nodo correcto segun el prefijo.



Transparencia de ubicacion

Las uniones distribuidas devuelven datos sin que el cliente sepa de varios nodos.



Atomicidad cross-site

Las transferencias confirman ambos saldos o ninguno.



Autonomia local

Ante caida de un nodo, las operaciones locales del otro siguen funcionando.

Extension: migracion a CockroachDB

Reto adicional para los equipos PFC: migrar el mismo esquema a un cluster CockroachDB de tres nodos.

- Complejidad del despliegue: docker-compose con 3 nodos versus 2 PostgreSQL
- Comportamiento ante caida: Raft sigue operando con 2 de 3 nodos
- Transparencia de fragmentacion: automatica via ranges, sin catalogo manual
- Serializabilidad estricta de CockroachDB vs aislamiento de PostgreSQL local
- Programar con JDBC: 100% compatible, sin cambios de codigo de aplicacion

Este reto es la base de la Guia GA-SUM-03 de la semana 11

Conclusiones

Cinco ideas para llevar a la vida profesional

1

BDD es cambio de paradigma, no evolucion

Introduce trade-offs ineludibles entre consistencia, disponibilidad y tolerancia a particiones. No es solo SQL en varios servidores.

2

Tres decisiones estrategicas iniciales

Arquitectura (cliente-servidor / federada / P2P), esquema de distribucion (fragmentacion + replicacion) y modelo de consistencia (CP / AP, fuerte / eventual).

3

Los protocolos sostienen las garantias

2PL, 2PC, 3PC, MVCC y consenso (Raft, Paxos) son la maquinaria que hace posible ACID en distribuido. Conocer sus limites es indispensable.

4

La teoria se materializa en codigo

Codigo Java compilable, contenedores Docker funcionales, comportamientos verificables. La distancia entre pizarra y produccion es corta pero exige rigor.

5

Eleccion tecnologica con consciencia

Spanner, CockroachDB, Cassandra, DynamoDB, MongoDB, PostgreSQL distribuido: cada uno resuelve un problema distinto. No hay sistema universalmente superior.

Referencias clave

Todas con DOI o ISBN verificado

- [1] M. T. Ozu and P. Valduriez, Principles of Distributed Database Systems, 4th ed. Cham: Springer, 2020. doi: 10.1007/978-3-030-26253-2
- [2] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA: O'Reilly Media, 2017. ISBN: 978-1-4493-7332-0
- [3] E. Brewer, "CAP twelve years later," IEEE Computer, vol. 45, no. 2, pp. 23-29, Feb. 2012. doi: 10.1109/MC.2012.37
- [4] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design," IEEE Computer, vol. 45, no. 2, pp. 37-42, Feb. 2012. doi: 10.1109/MC.2012.33
- [5] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in Proc. USENIX ATC, 2014, pp. 305-319. ISBN: 978-1-931971-10-2
- [6] M. van Steen and A. S. Tanenbaum, Distributed Systems, 4th ed. Maarten van Steen, 2025. ISBN: 978-90-815406-3-6

Material complementario disponible

Documento LaTeX integral (34 paginas) + Guia GA-SUM-03 +Codigo Java completo del Banco del Austro

Gracias por su atencion -- preguntas y discusion